

UNIVERSIDADE VEIGA DE ALMEIDA
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

THIAGO BARBOSA DA SILVA BASTOS

ÁRVORE BINÁRIA DE BUSCA EM C

Rio de Janeiro
2025

THIAGO BARBOSA DA SILVA BASTOS

ÁRVORE BINÁRIA DE BUSCA EM C

Trabalho acadêmico apresentado à disciplina de Estrutura de Dados, do curso de Análise e Desenvolvimento de Sistemas da Universidade Veiga de Almeida, como atividade avaliativa AVA II.

Rio de Janeiro
2025

SUMÁRIO

1 INTRODUÇÃO.....	1
2 FUNDAMENTAÇÃO TEÓRICA.....	1
2.1 Estrutura de uma árvore binária de busca.....	1
2.2 Percursos em profundidade.....	2
3 DESENVOLVIMENTO DA SOLUÇÃO.....	2
3.1 Estrutura do nó.....	2
3.2 Inserção de valores.....	3
3.3 Remoção de nós.....	3
3.4 Percursos e interface do usuário.....	4
3.5 Validação de entrada e gerenciamento de memória.....	4
4 TESTES E RESULTADOS.....	5
5 CONCLUSÃO.....	6
REFERÊNCIAS.....	6

1 INTRODUÇÃO

Este trabalho apresenta o desenvolvimento de um programa em linguagem C para manipular uma árvore binária de busca. A solução foi elaborada a partir do enunciado da atividade AVA II da disciplina de Estrutura de Dados, que solicitou a implementação de um menu com operações de inclusão, remoção e três formas de percurso em profundidade.

A árvore binária de busca organiza valores por meio de uma relação hierárquica: valores menores são posicionados na subárvore esquerda e valores maiores na subárvore direita. Essa propriedade orienta as operações de inserção e remoção e permite que o percurso em ordem apresente os elementos em sequência crescente.

A versão revisada do programa também incorpora validação de entradas, tratamento de valores duplicados, mensagens coerentes para operações sem sucesso e liberação integral da memória alocada. Com isso, o projeto demonstra não apenas o funcionamento da estrutura de dados, mas também práticas de programação defensiva e organização modular do código.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Estrutura de uma árvore binária de busca

Uma árvore binária é composta por nós, e cada nó pode possuir até dois descendentes: um filho à esquerda e outro à direita. Em uma árvore binária de busca, os valores são organizados segundo uma regra de ordenação. Para cada nó, todos os valores da subárvore esquerda devem ser menores, enquanto os valores da subárvore direita devem ser maiores.

No programa desenvolvido, cada nó armazena um valor inteiro e dois ponteiros. Esses ponteiros conectam o nó às suas subárvores. Como os nós são criados somente quando necessários, a estrutura utiliza alocação dinâmica de memória.

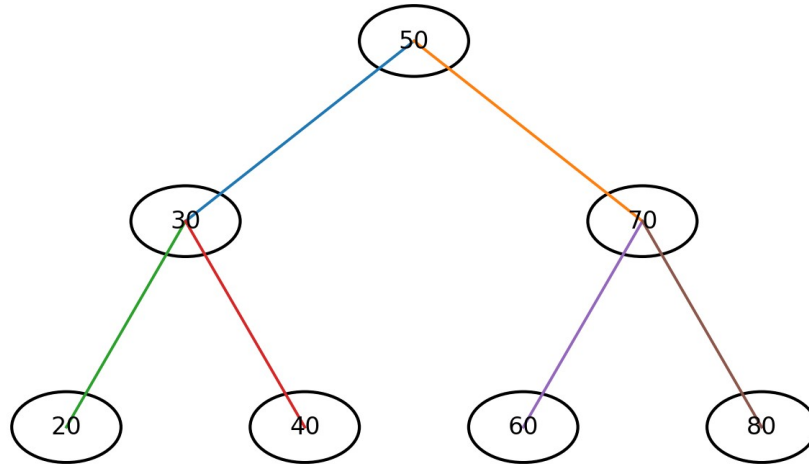


Figura 1 - Exemplo de árvore binária de busca utilizada nos testes

Na árvore representada, 50 é a raiz. Os valores 30, 20 e 40 pertencem à subárvore esquerda, enquanto 70, 60 e 80 pertencem à subárvore direita. A mesma regra de ordenação é aplicada recursivamente em cada subárvore.

2.2 Percursos em profundidade

Os percursos em profundidade visitam todos os nós da árvore, mas alteram a posição em que a raiz é processada. O programa oferece três formas de percurso:

- Pré-ordem: visita a raiz, depois a subárvore esquerda e, por último, a subárvore direita.
- Em ordem: visita a subárvore esquerda, depois a raiz e, por último, a subárvore direita.

Em uma árvore binária de busca, esse percurso exibe os valores em ordem crescente.

- Pós-ordem: visita a subárvore esquerda, depois a subárvore direita e, por último, a raiz.

3 DESENVOLVIMENTO DA SOLUÇÃO

3.1 Estrutura do nó

A estrutura No define a unidade básica da árvore. Além do valor inteiro, cada nó possui ponteiros para os filhos esquerdo e direito. Quando um novo nó é criado, ambos os ponteiros são inicializados com NULL, indicando que ainda não existem descendentes.

Trecho 1 - Estrutura do nó

```
typedef struct No {
    int valor;
    struct No *esquerda;
    struct No *direita;
} No;
```

3.2 Inserção de valores

A função `inserir_no` percorre a árvore comparando o valor recebido com o valor de cada nó. Quando encontra uma ligação vazia, aloca um novo nó e o conecta à árvore. Se o valor já existir, a operação é recusada, evitando duplicidades.

Trecho 2 - Inserção e tratamento de valores duplicados

```
static ResultadoInsercao inserir_no(No **raiz, int valor) {
    No **ligacao_atual = raiz;

    while (*ligacao_atual != NULL) {
        if (valor < (*ligacao_atual)->valor) {
            ligacao_atual = &(*ligacao_atual)->esquerda;
        } else if (valor > (*ligacao_atual)->valor) {
            ligacao_atual = &(*ligacao_atual)->direita;
        } else {
            return INSERCAO_DUPLICADA;
        }
    }

    No *novo = malloc(sizeof *novo);
    if (novo == NULL) {
        return INSERCAO_SEM_MEMORIA;
    }

    novo->valor = valor;
    novo->esquerda = NULL;
    novo->direita = NULL;
    *ligacao_atual = novo;

    return INSERCAO_SUCESSO;
}
```

3.3 Remoção de nós

A remoção exige o tratamento de três situações. Quando o nó não possui filho esquerdo, ele pode ser substituído diretamente pelo filho direito. Quando não possui filho direito, é substituído pelo filho esquerdo. Se possuir dois filhos, o programa localiza o menor valor da subárvore direita, copia esse valor para o nó-alvo e remove o nó sucessor.

Trecho 3 - Tratamento dos casos de remoção

```
if (alvo->esquerda == NULL) {
    *ligacao_atual = alvo->direita;
    free(alvo);
} else if (alvo->direita == NULL) {
    *ligacao_atual = alvo->esquerda;
    free(alvo);
} else {
    No **ligacao_sucessor = &alvo->direita;

    while ((*ligacao_sucessor)->esquerda != NULL) {
        ligacao_sucessor = &(*ligacao_sucessor)->esquerda;
    }

    No *sucessor = *ligacao_sucessor;
    alvo->valor = sucessor->valor;
    *ligacao_sucessor = sucessor->direita;
    free(sucessor);
}
```

3.4 Percursos e interface do usuário

Os três percursos foram implementados de forma recursiva. Cada função verifica se o nó atual existe e, em seguida, processa a raiz e as subárvores na ordem correspondente. O menu permite selecionar inclusão, remoção, pré-ordem, em ordem, pós-ordem ou encerramento.

Trecho 4 - Percurso em ordem

```
static void percorrer_em_ordem(const No *raiz) {
    if (raiz == NULL) {
        return;
    }

    percorrer_em_ordem(raiz->esquerda);
    printf("%d ", raiz->valor);
    percorrer_em_ordem(raiz->direita);
}
```

3.5 Validação de entrada e gerenciamento de memória

A leitura de opções e valores foi implementada com `fgets` e `strtol`. Essa abordagem permite rejeitar caracteres inválidos, textos misturados com números e valores fora do intervalo do tipo `int`. O programa continua solicitando uma entrada até receber um número inteiro válido ou até que a entrada seja encerrada.

Antes de finalizar, a função `liberar_arvore` percorre as subárvores em pós-ordem, libera cada nó com `free` e redefine os ponteiros para `NULL`. Esse procedimento evita que a memória alocada permaneça sem liberação após o término do programa.

Trecho 5 - Liberação da memória

```
static void liberar_arvore(No **raiz) {  
    if (raiz == NULL || *raiz == NULL) {  
        return;  
    }  
  
    liberar_arvore(&(*raiz)->esquerda);  
    liberar_arvore(&(*raiz)->direita);  
    free(*raiz);  
    *raiz = NULL;  
}
```

4 TESTES E RESULTADOS

A versão revisada foi compilada com o padrão C11 e opções de verificação do compilador. A execução de testes utilizou os valores 50, 30, 70, 20, 40, 60 e 80, resultando na árvore mostrada na Figura 1.

Teste	Entrada ou ação	Resultado observado
Inserção dos valores	50, 30, 70, 20, 40, 60 e 80	Todos os valores foram incluídos.
Percurso em pré-ordem	Opção 3	50 30 20 40 70 60 80
Percurso em ordem	Opção 4	20 30 40 50 60 70 80
Percurso em pós-ordem	Opção 5	20 40 30 60 80 70 50
Valor duplicado	Nova inclusão do valor 40	Operação recusada com mensagem apropriada.
Remoção inexistente	Remoção do valor 999	Valor não encontrado; árvore preservada.
Remoção com dois filhos	Remoção do valor 50	Raiz substituída pelo sucessor; estrutura mantida.
Entrada inválida	Texto "abc" no menu	Entrada rejeitada e nova leitura solicitada.
Encerramento	Opção 0	Todos os nós liberados da memória.

Comando de compilação utilizado:

```
cc -std=c11 -Wall -Wextra -Wpedantic -Wconversion -Wshadow -Werror arvore.c -o arvore
```

A compilação foi concluída sem avisos. Os resultados confirmam que a implementação mantém a propriedade da árvore binária de busca, distingue operações bem-sucedidas das operações recusadas e trata entradas inválidas sem interromper o programa.

5 CONCLUSÃO

O projeto atendeu ao objetivo de implementar, em linguagem C, uma árvore binária de busca com inclusão, remoção e percursos em pré-ordem, em ordem e pós-ordem. A solução utiliza estruturas, ponteiros, alocação dinâmica e recursividade para representar e manipular dados hierárquicos.

A revisão do código acrescentou validação de entradas, tratamento de duplicidades, mensagens de resultado mais precisas e liberação completa da memória. Os testes demonstraram o funcionamento correto dos percursos e da remoção, inclusive no caso de um nó com dois filhos. Dessa forma, o trabalho reúne os conceitos centrais da disciplina e apresenta uma implementação mais segura, organizada e adequada para publicação em portfólio acadêmico.

REFERÊNCIAS

UNIVERSIDADE VEIGA DE ALMEIDA. Estrutura de Dados: AVA II - busca em árvores binárias. Rio de Janeiro, 2025. Material didático.